

Tarefa 1 — Multiplicação Matriz × Vetor: acesso por linha vs. por coluna

Objetivo

Implementar duas versões do produto MxV em C e observar como a **ordem em que percorremos a matriz** influencia o tempo de execução, relacionando os resultados ao uso da memória cache.

O que foi implementado

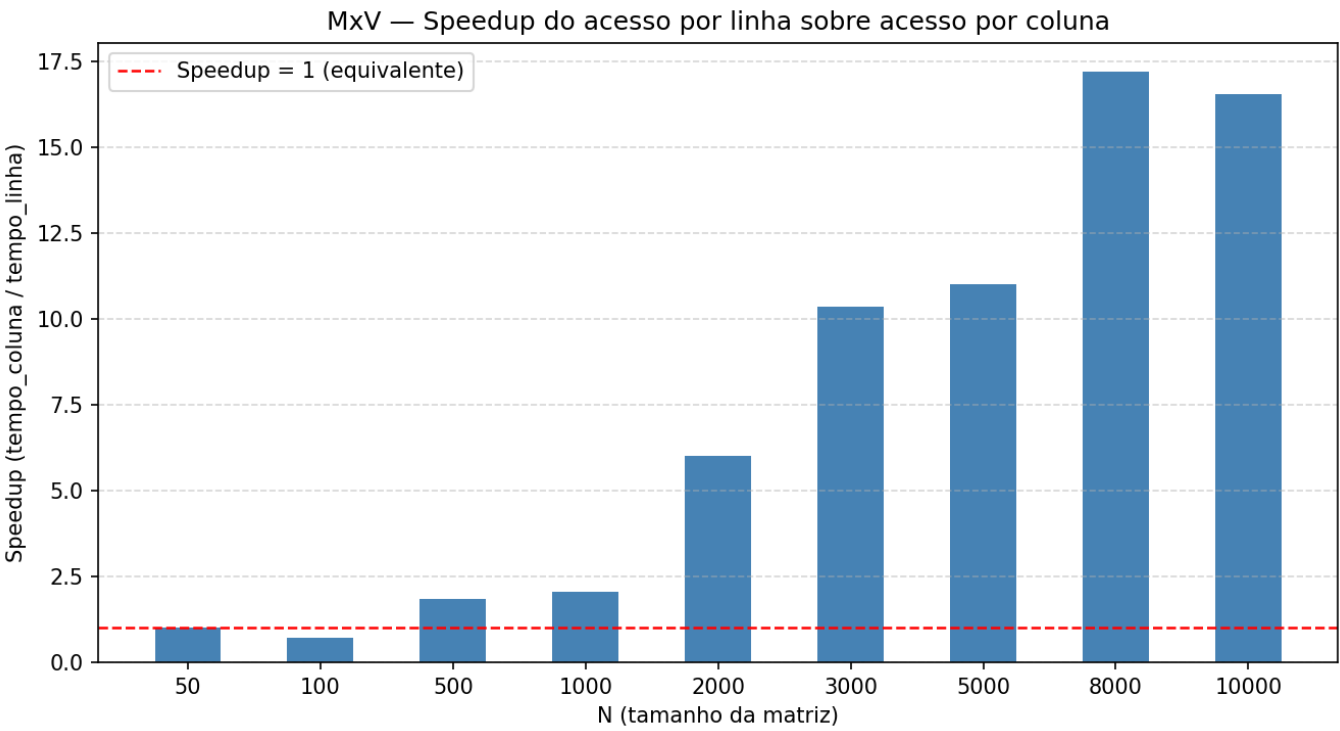
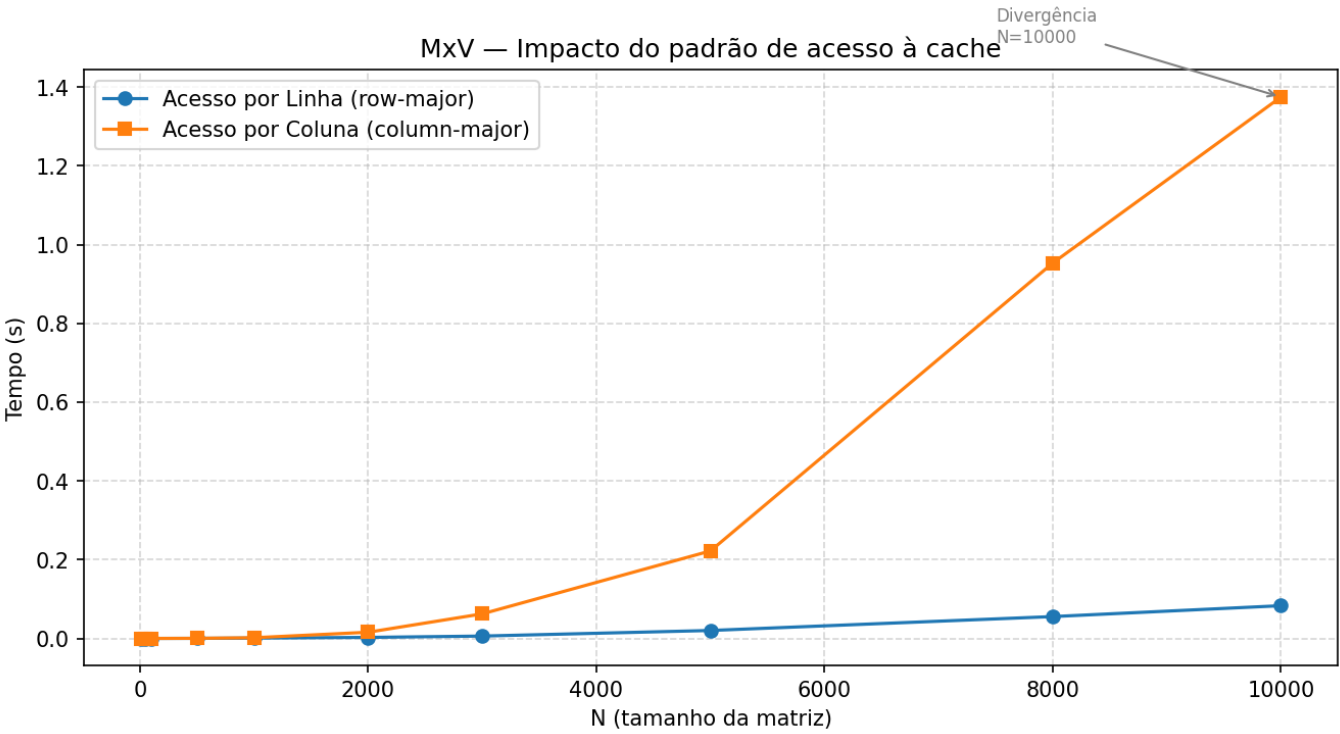
Foram criadas duas versões que fazem exatamente o mesmo cálculo — multiplicar uma matriz A por um vetor B. A única diferença é a ordem dos loops:

- **Por linha:** loop externo percorre linhas, interno percorre colunas
 - **Por coluna:** loop externo percorre colunas, interno percorre linhas
-

Resultados

N	Linha (s)	Coluna (s)	Coluna é mais lenta por
10	~0	~0	—
50	0.000001	0.000001	~1×
100	0.000007	0.000005	~0.7× ¹
500	0.000157	0.000292	1.9×
1000	0.000876	0.001809	2.1×
2000	0.002621	0.015740	6.0×
3000	0.006016	0.062340	10.4×
5000	0.020186	0.222115	11.0×
8000	0.055477	0.952994	17.2×
10000	0.083098	1.374537	16.5×

¹ Em N=100 os tempos estão na escala de microssegundos — a inversão é ruído de medição, não um resultado real.



Por que a diferença acontece

Row-major: como C organiza matrizes na memória

Em C, matrizes são armazenadas em **row-major order** — linha por linha. Uma matriz `A[3][3]` ocupa a memória assim:

Posição:	0	1	2	3	4	5	6	7
8								
Dado:	<code>A[0][0]</code>	<code>A[0][1]</code>	<code>A[0][2]</code>	<code>A[1][0]</code>	<code>A[1][1]</code>	<code>A[1][2]</code>	<code>A[2][0]</code>	<code>A[2][1]</code>
<code>A[2][2]</code>								
	--- linha 0 ---			--- linha 1 ---			--- linha 2 ---	

Os elementos de uma mesma linha ficam contíguos. Os elementos de uma mesma coluna estão separados por toda uma linha de distância.

Acesso por linha — a cache trabalha bem

Quando o loop interno percorre colunas (`j` variando, `i` fixo), acessamos `A[i][0]`, `A[i][1]`, `A[i][2]`... — exatamente na ordem em que estão na memória. Quando a cache carrega um trecho da matriz para atender ao primeiro acesso, os próximos elementos já estão lá. A cache é aproveitada ao máximo e os acessos à RAM são mínimos.

Acesso por coluna — a cache é constantemente invalidada

Quando o loop interno percorre linhas (`i` variando, `j` fixo), acessamos `A[0][j]`, `A[1][j]`, `A[2][j]`... — que na memória estão separados por uma linha inteira. Cada acesso cai em uma região da memória completamente diferente. A cache troca o conteúdo a cada iteração, e o processador precisa ir à RAM repetidamente para buscar dados que nunca estão onde ele esperava.

Por que os tempos só divergem a partir de $N \approx 1000$

Quando a matriz cabe inteira na cache, ela é carregada da RAM uma única vez no início — depois disso, todos os acessos são atendidos pela cache independentemente da ordem. É por isso que para N pequeno as duas versões são quase iguais.

À medida que N cresce, a matriz passa a ser maior do que a cache consegue guardar. A partir desse ponto, a ordem de acesso passa a determinar quantas vezes o processador precisa recorrer à RAM:

Faixa de N	Comportamento
$N \leq 500$	Matriz cabe na cache — ambas as versões igualmente rápidas
N entre 1000 e 3000	Matriz começa a exceder a cache — diferença cresce rapidamente ($\sim 2\text{--}10\times$)
$N \geq 5000$	Matriz muito maior que a cache — versão por coluna vai à RAM em quase toda iteração ($11\text{--}17\times$ mais lenta)

Por que o speedup estabiliza entre $N=8000$ e $N=10000$

Em $N=8000$, o speedup atinge o pico de **17.2x**. Em $N=10000$, cai levemente para **16.5x** — uma queda muito menor do que a medição anterior sugeria (que ia de 19.3x para 14.6x). Isso indica que, nessa faixa, ambas as versões já estão saturando o barramento de memória: a versão por coluna continua sofrendo cache misses em quase toda iteração, e a versão por linha, mesmo com acesso sequencial, já não consegue esconder completamente a latência de um vetor de ~800 MB. O speedup se estabiliza porque os dois programas passam a ser igualmente limitados pela largura de banda da RAM.

Conclusão

Dois programas com o mesmo resultado matemático chegaram a ter **17x de diferença no tempo de execução** (pico em $N=8000$). A causa é unicamente a ordem de acesso à memória. Em C, respeitar o layout row-major — percorrer matrizes com o índice de coluna variando no loop mais interno — é essencial para manter a cache eficiente e evitar acessos desnecessários à RAM.

Código

mult_por_linha.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main(int argc, char *argv[]) {
    int N = (argc > 1) ? atoi(argv[1]) : 1000;

    double *A = malloc((size_t)N * N * sizeof(double));
    double *B = malloc(N * sizeof(double));
    double *C = malloc(N * sizeof(double));

    if (!A || !B || !C) {
        fprintf(stderr, "Erro de alocação de memória\n");
        return 1;
    }

    srand(42);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
            A[i * N + j] = (double)rand() / RAND_MAX;
        B[i] = (double)rand() / RAND_MAX;
        C[i] = 0.0;
    }

    double start = omp_get_wtime();

    // linha externa, coluna interna – acesso row-major (cache-friendly)
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            C[i] += A[i * N + j] * B[j];

    double end = omp_get_wtime();

    printf("%d,%.6f\n", N, end - start);

    if (C[0] < -1e300) printf("dummy\n");

    free(A); free(B); free(C);
    return 0;
}
```

mult_por_coluna.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```

```
int main(int argc, char *argv[]) {
    int N = (argc > 1) ? atoi(argv[1]) : 1000;

    double *A = malloc((size_t)N * N * sizeof(double));
    double *B = malloc(N * sizeof(double));
    double *C = malloc(N * sizeof(double));

    if (!A || !B || !C) {
        fprintf(stderr, "Erro de alocação de memória\n");
        return 1;
    }

    srand(42);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
            A[i * N + j] = (double)rand() / RAND_MAX;
        B[i] = (double)rand() / RAND_MAX;
        C[i] = 0.0;
    }

    double start = omp_get_wtime();

    // coluna externa, linha interna – acesso column-major (cache-unfriendly)
    for (int j = 0; j < N; j++)
        for (int i = 0; i < N; i++)
            C[i] += A[i * N + j] * B[j];

    double end = omp_get_wtime();

    printf("%d,%.6f\n", N, end - start);

    if (C[0] < -1e300) printf("dummy\n");

    free(A); free(B); free(C);
    return 0;
}
```

Compilação e execução

```
gcc -O2 -fopenmp mult_por_linha.c -o linha -lm
gcc -O2 -fopenmp mult_por_coluna.c -o coluna -lm

./linha 5000
./coluna 5000
```